

Compression de texte sans perte :

Codes de Huffman

e2502096 Matthias DUMAS

e2502197 Lise RENAUD

L'algorithme de Huffman est une méthode de compression sans perte fondée sur le codage à longueur variable. Dans ce TP, nous avons implémenté cet algorithme selon trois approches différentes: statique, classique et adaptative/streaming. Chaque approche utilise une manière différente de construire et de mettre à jour l'arbre de Huffman pour optimiser la compression des données.

Huffman statique

La première version implémentée repose sur un algorithme de Huffman statique. L'arbre de Huffman est construit à partir d'une table de fréquences prédéfinie et commune à l'émetteur et au récepteur. Cette table représente les fréquences d'apparition des caractères dans des textes rédigés en français. L'intérêt de la version statique est que l'arbre n'a pas besoin d'être stocké dans le fichier compressé. Lors de la compression comme de la décompression, les deux parties reconstruisent exactement le même arbre à partir du dictionnaire partagé. Cela réduit la taille des données compressées puisque seule la suite de bits encodée est transmise.

La construction de l'arbre de Huffman s'appuie sur une file de priorité implémentée avec la bibliothèque `heapq` de Python. Chaque caractère est représenté par un nœud contenant son symbole et sa fréquence. L'algorithme extrait ensuite les deux nœuds de plus faible fréquence afin de les fusionner dans un nouveau nœud parent. Ce processus est répété jusqu'à l'obtention de la racine de l'arbre. Une fois l'arbre construit, les codes binaires sont générés par un parcours récursif. Chaque déplacement vers le fils gauche ajoute un bit 0, tandis qu'un déplacement vers le fils droit ajoute un bit 1. Les feuilles de l'arbre correspondent aux caractères du dictionnaire.

La base du dictionnaire proposé ne traitant que l'alphabet [a-z] sans les accents, nous avons donc ajouté quelques caractères supplémentaires (accents, ponctuation). Cependant, comme tous les caractères UTF-8 ne peuvent pas être intégrés dans l'arbre sans augmenter fortement sa taille, nous avons ajouté un caractère d'échappement `ESC`. Lorsqu'un caractère inconnu est rencontré, le programme encode d'abord le code Huffman associé à `ESC`, puis encode directement les octets UTF-8 du caractère sur 8 bits. Lors de la décompression, ce symbole spécial indique qu'il faut lire les octets suivants afin de reconstruire le caractère original. Cette solution permet de conserver un arbre compact tout en garantissant une restitution fidèle du texte initial.

Les données compressées étant manipulées sous forme de chaînes de bits, une conversion en octets est nécessaire avant l'écriture dans le fichier de sortie. Un système de padding a été ajouté afin de compléter le dernier octet lorsque le nombre de bits n'est pas multiple de huit. Le nombre de bits ajoutés est stocké dans le premier octet du fichier compressé.

Lors de la décompression, le programme reconstruit d'abord l'arbre de Huffman à partir de la table de fréquences partagée entre l'émetteur et le récepteur. Les données binaires du fichier compressé sont ensuite reconverties en une chaîne de bits, en retirant les bits de padding ajoutés lors de la compression. Le décodage s'effectue en parcourant l'arbre bit par bit : un 0 correspond à un déplacement vers le fils gauche et un 1 vers le fils droit. Lorsqu'une feuille est atteinte, le caractère associé est ajouté au texte reconstruit. Si le symbole spécial `ESC` est rencontré, les bits suivants sont lus par groupes de 8 afin de reconstruire le caractère UTF-8 original.

Huffman classique

Huffman classique repose sur la construction d'un arbre binaire optimal à partir des fréquences des caractères présents dans le fichier d'entrée. Dans cette version, l'ensemble du fichier est d'abord lu afin de calculer les fréquences de chaque octet. Cela permet de générer un arbre de Huffman fixe, partagé entre l'émetteur et le récepteur. Avec la classe *Node*, chaque nœud contient un caractère (ou *None* pour les nœuds internes), sa fréquence, ainsi que des pointeurs vers ses fils gauche et droit. La méthode `is_leaf()` permet d'identifier les feuilles, c'est-à-dire les nœuds contenant un caractère réel.

Pour construire l'arbre, l'algorithme utilise une file de priorité (*heapq*), ce qui permet l'extraction des deux nœuds les moins fréquents à chaque étape. Chaque caractère est d'abord transformé en nœud, puis inséré dans la file de priorité. L'algorithme fusionne ensuite les deux nœuds de plus faible fréquence en un nœud parent, dont la fréquence est la somme des deux enfants. Ce processus se répète jusqu'à ce qu'il ne reste plus qu'un seul nœud, qui devient la racine de l'arbre de Huffman. Cette approche garantit une construction optimale de l'arbre, conformément à l'algorithme classique.

Une fois l'arbre construit, la table des codes est générée via un parcours récursif. La fonction `build_codes()` parcourt l'arbre en ajoutant un 0 lorsqu'elle descend à gauche et un 1 lorsqu'elle descend à droite. Lorsqu'une feuille est atteinte, le code binaire associé au caractère est enregistré. Cette table de correspondance permet ensuite d'encoder efficacement le fichier d'entrée en une longue chaîne de bits.

L'encodage consiste à remplacer chaque caractère du fichier par son code binaire. La chaîne de bits obtenue est ensuite convertie en bytes grâce à la fonction `pack()`, qui ajoute du *padding* si nécessaire pour obtenir un alignement sur 8 bits. Le fichier compressé contient :

- le nombre de bits de padding,
- le nombre de caractères distincts,
- la table des fréquences (caractère + fréquence),
- les données compressées.

La décompression suit le processus inverse : reconstruction de l'arbre à partir des fréquences stockées, conversion des octets en bits, puis parcours de l'arbre pour retrouver les caractères originaux. Chaque bit guide la descente dans l'arbre jusqu'à atteindre une feuille, ce qui permet de reconstituer le fichier initial. Cette implémentation respecte le

fonctionnement du Huffman classique : l'arbre est construit une seule fois à partir du fichier complet, puis transmis au décodeur.

Huffman streaming

Huffman adaptatif, également appelé Huffman streaming, ne construit pas l'arbre avant la compression. L'arbre évolue dynamiquement au fur et à mesure de la lecture du flux de données. Cette approche permet à l'émetteur et au récepteur de maintenir simultanément le même arbre sans avoir besoin de transmettre une table de fréquences ni la structure complète de l'arbre dans le fichier compressé. Les deux côtés reconstruisent progressivement la même structure uniquement à partir des données échangées.

Nous nous sommes donc inspirés de l'algorithme FGK (Faller-Gallager-Knuth). Chaque nœud de l'arbre contient plusieurs informations : le symbole associé, son poids, ses liens parent-enfant ainsi qu'un numéro d'ordre utilisé pour garantir les propriétés de l'arbre adaptatif lors des échanges de nœuds.

Au démarrage, l'arbre contient uniquement un nœud spécial appelé NYT (*“Not Yet Transmitted”*). Lorsqu'un symbole apparaît pour la première fois dans le flux, le programme encode d'abord le chemin vers ce nœud NYT, puis transmet directement le symbole sur 8 bits. Un nouveau nœud feuille est ensuite créé dans l'arbre pour représenter ce symbole. Si le symbole a déjà été rencontré auparavant, seul son code courant dans l'arbre est transmis.

Après chaque lecture ou écriture de symbole, l'arbre de Huffman adaptatif est mis à jour dynamiquement afin d'enregistrer les nouvelles fréquences observées. Le poids du nœud correspondant au symbole traité est incrémenté, puis cette modification se propage vers ses parents jusqu'à la racine. Pour garantir que l'arbre conserve les propriétés de l'algorithme FGK, certains nœuds peuvent être échangés grâce à la méthode `swap()`. Cette opération permet de maintenir l'ordre des nœuds selon leur poids et leur position hiérarchique, évitant toute modification de la structure. La fonction `update()` parcourt l'arbre en remontant depuis le nœud modifié, identifie le nœud le plus haut dans le bloc de poids équivalent (`find_highest_in_block()`), et on effectue un échange si nécessaire. Chaque incrément de poids est ensuite enregistré dans la structure `blocks`, qui regroupe les nœuds par poids pour accélérer la recherche et la mise à jour.

Nous avons utilisé une structure de blocs de poids pour optimiser la recherche et la mise à jour de l'arbre. Les nœuds sont regroupés dans une structure de type `defaultdict(set)` selon leur poids courant, ce qui permet de maintenir dynamiquement des ensembles de nœuds partageant la même fréquence. Lorsqu'un nœud voit son poids incrémenté, la fonction `update_blocks()` déplace ce nœud du bloc correspondant à son ancien poids vers celui de son nouveau poids. Cette organisation facilite la recherche du nœud ayant la priorité la plus importante dans un bloc de poids donné grâce à la méthode `find_highest_in_block()`, utilisée lors des échanges (`swap()`) pour préserver la propriété de numérotation décroissante imposée par l'algorithme FGK.

Afin d'assurer la compatibilité UTF-8, le texte est converti en séquence d'octets avant compression. L'algorithme travaille donc directement sur les *bytes* plutôt que sur les

caractères Unicode. Cette méthode garantit une reconstruction exacte du texte original, quel que soit le contenu du fichier. Comme dans les autres versions, les données compressées sont finalement converties en octets afin d'être écrites dans un fichier binaire. Un en-tête contenant le nombre de bits de *padding* est ajouté pour permettre une reconstruction correcte des données lors de la décompression.

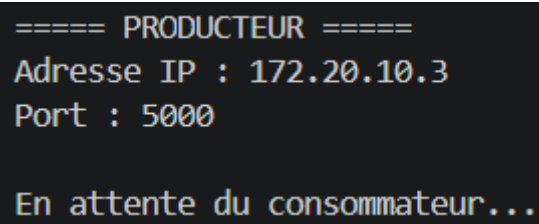
La décompression s'effectue de manière synchronisée avec la compression. Le décodeur initialise le même arbre contenant uniquement le nœud NYT, puis lit progressivement les bits du flux compressé. Pour chaque symbole décodé, l'arbre est mis à jour exactement de la même manière que côté compression, ce qui permet au récepteur de reconstruire dynamiquement la même structure sans transmission explicite de l'arbre. Lorsqu'un nœud NYT est rencontré, les 8 bits suivants sont interprétés comme un nouveau symbole UTF-8 qui est ajouté à l'arbre. Les symboles déjà connus sont directement retrouvés en parcourant l'arbre selon les bits lus.

Démonstration du Huffman streaming

Sur un terminal, on lance la commande :

```
python3 /path/to/huffman-streaming-demo.py -p /path/to/input.txt
```

On observe ensuite l'adresse IP et le port :

A terminal window with a dark background and light-colored text. The text displayed is: ===== PRODUCTEUR =====, Adresse IP : 172.20.10.3, Port : 5000, and En attente du consommateur...

```
===== PRODUCTEUR =====  
Adresse IP : 172.20.10.3  
Port : 5000  
  
En attente du consommateur...
```

Sur un autre terminal, nous lançons la commande pour démarrer le consommateur en précisant l'IP et le port :

```
python3 /path/to/huffman-streaming-demo.py -c 172.20.10.3:5000
```

Nous pouvons alors observer lors de la transmission des données que l'arbre se met à jour à chaque nouveau caractère. Nous avons mis en place l'affichage de l'arbre après avoir transmis 50 nouveaux caractères. Afin d'observer plus facilement le phénomène, nous ajoutons une pause de 0.0001 seconde dans le programme entre chaque caractère. Ces valeurs peuvent être modifiées dans le code source.

A la fin on vérifie que le consommateur obtient le même texte que celui transmis par le producteur. Voici un exemple avec le fichier `le-horla.txt` :

Et j'allai me cacher au fond de mon jardin, dans un massif de lauriers. Comme ce fut long ! comme ce fut long ! Tout était noir, muet, immobile ; pas un souffle d'air, pas une étoile, des montagnes de nuages qu'on ne voyait point, mais qui pesaient sur mon âme si lourds, si lourds.

Je regardais ma maison, et j'attendais. Comme ce fut long ! Je croyais déjà que le feu s'était éteint tout seul, ou qu'il l'avait éteint, lui, quand une des fenêtres d'en bas creva sous la poussée de l'incendie, et une flamme, une grande flamme rouge et jaune, longue, molle, caressante, monta le long du mur blanc et le baisa jusqu'au toit. Une lueur courut dans les arbres, dans les branches, dans les feuilles, et un frisson, un frisson de peur aussi. Les oiseaux se réveillaient ; un chien se mit à hurler ; il me sembla que le jour se levait ! Deux autres fenêtres éclatèrent aussitôt, et je vis que tout le bas de ma demeure n'était plus qu'un effrayant brasier.

Mais un cri, un cri horrible, suraigu, déchirant, un cri de femme passa dans la nuit, et deux mansardes s'ouvrirent ! J'avais oublié mes domestiques ! Je vis leurs faces affolées, et leurs bras qui s'agitaient !...

Alors, éperdu d'horreur, je me mis à courir vers le village en hurlant : « Au secours ! au secours ! au feu ! au feu ! » Je rencontrai des gens qui s'en venaient déjà et je retournai avec eux, pour voir !

La maison, maintenant, n'était plus qu'un bûcher horrible et magnifique, un bûcher monstrueux, éclairant toute la terre, un bûcher où brûlaient des hommes, et où il brûlait aussi, lui, lui, mon prisonnier, l'être nouveau, le nouveau maître, le Horla !

Soudain le toit tout entier s'engloutit entre les murs, et un volcan de flammes jaillit jusqu'au ciel. Par toutes les fenêtres ouvertes sur la fournaise, je voyais la cuve de feu, et je pensais qu'il était là, dans ce four, mort...

« Mort ? Peut-être ?... Son corps ? son corps que le jour traversait n'était-il pas indestructible par les moyens qui tuent les nôtres ?

« S'il n'était pas mort ?... seul peut-être le temps a pris sur l'être Invisible et Redoutable. Pourquoi ce corps transparent, ce corps inconnaissable, ce corps d'Esprit, s'il devait craindre, lui aussi, les maux, les blessures, les infirmités, la destruction prématurée ?

« La destruction prématurée ? toute l'épouvante humaine vient d'elle ! Après l'homme, le Horla. – Après celui qui peut mourir tous les jours, à toutes les heures, à toutes les minutes, par tous les accidents, est venu celui qui ne doit mourir qu'à son jour, à son heure, à sa minute, parce qu'il a touché la limite de son existence !

« Non... non... sans aucun doute, sans aucun doute... il n'est pas mort... Alors... alors... il va donc falloir que je me tue, moi !... »

===== FIN CONSUMMATEUR =====
Temps total : 80.4549 secondes

Comparaison des trois implémentations

Afin de comparer nos différentes implémentations, nous avons mesuré différentes performances comme le temps de compression ou de décompression et le taux de compression sur différentes tailles de fichiers.

Cas d'usage	Statique	Classique	Streaming
Fichier le-horla.txt (55ko)			
Taux de compression (réduction de...)	40,5% (33ko)	42% (32ko)	42% (32ko)
Encodage	0,0226s	0,0481s	0,8877s
Décodage	0,1124s	0,1002s	0,6902s
Fichier 4,2Mo			
Taux de compression (réduction de...)	34,64% (2,8Mo)	42,09% (2,5Mo)	42,7% (2,4Mo)
Encodage	2,1316s	1,8249s	64,26s
Décodage	5,7920s	3,3802s	60,96s

On observe que l'implémentation **statique** présente un taux de compression plus faible et, sur un fichier volumineux, une vitesse de compression moyenne. Elle reste néanmoins très efficace pour des fichiers de petite taille.

L'implémentation **classique** affiche un taux de compression d'environ 42 % et se montre rapide, aussi bien sur de petits que sur de grands fichiers.

Enfin, l'implémentation **streaming** atteint également un taux de compression de 42 %, quelle que soit la taille du fichier. Ses performances sont plus lentes, car l'arbre de Huffman est mis à jour dynamiquement en temps réel.

Recommandation des utilisations adaptées pour chaque cas d'usage

Le Huffman statique est idéal pour les fichiers dont la distribution de caractères est connue à l'avance, par exemple des textes dont la langue est fixée. Son avantage est qu'il ne comporte pas d'*overhead* : il ne stocke pas d'arbre dans le fichier compressé, seulement le texte d'origine. Cependant, lorsque l'on s'éloigne de la table de référence fixée, cette méthode est peu efficace. Aussi, il est nécessaire que l'émetteur et le récepteur partagent la même table afin d'encoder et décoder les données.

Le Huffman classique est le choix naturel pour la compression de fichiers isolés, dont on ne connaît pas à l'avance la distribution des caractères. L'arbre étant construit sur mesure à partir du contenu réel, il offre un taux de compression optimal pour chaque fichier. Le coût est un *overhead* fixe correspondant à l'encodage de l'arbre dans le fichier compressé, ce qui le rend moins intéressant pour de très petits fichiers, mais tout à fait pertinent dès que la taille du contenu dépasse quelques kilo-octets.

Le Huffman streaming met à jour son arbre dynamiquement, au fur et à mesure de la lecture des caractères. Bien que plus lente, cette approche est parfaitement adaptée aux flux continus ou aux données reçues en temps réel. Elle est recommandée pour les cas de streaming, les capteurs IoT, ou les communications réseau, où les données arrivent progressivement et nécessitent une adaptation constante. Aussi, en début de flux, le taux de compression n'est pas optimal tant que l'arbre n'a pas convergé.